Exploratory Data Analysis (EDA) Report on Loan Decisions

Candidate Number : 886132

Module Code : BBM126 - Programming for Data Analytics

Module Leader : Muhidin Mohamed & Shubhadeep Mukherjee

Date of Submission: 29th June 2026

Introduction

This report represent a dataset (loanapp.csv) containing mortgage applications across 17 different variables. The data tracks basic applicant details like race, gender, and marital status right alongside financial figures like income, requested loan size, home purchase price, and credit records. We begin the project by summarizing the data, identifying and managing any missing values, and displaying the primary distributions. The link between the variables is then examined using a variety of statistical tests. Additionally, data grouping and subsetting are done to better understand particular demographic trends. Lastly, a linear regression model is created to determine how many factors together effect. To determine how different factors in the data set impact the loan decision, all of these procedures and statistical tests were carried out.

1. Descriptive Statistics for the Dataset.

In [1]:
```
#Loading necessary packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
#Reading the data
df = pd.read_csv("loanapp.csv")
```

In [3]:
```
#Summarizing the data
print("loanapp.csv")
```

```
display(df.head())
```

loanapp.csv

	married	race	loan_decision	occupancy	loan_amount	applicant_income	nu
0	True	white	reject	1	128	74	
1	False	white	approve	1	128	84	
2	True	white	approve	1	66	36	
3	True	white	approve	1	120	59	
4	False	white	approve	1	111	63	

In [8]:
```
#Data categories
print("Categories and counts:")
df.info()
```

```
Categories and counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1988 entries, 0 to 1987
Data columns (total 17 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   married                  1985 non-null   object
 1   race                     1988 non-null   object
 2   loan_decision            1988 non-null   object
 3   occupancy                1988 non-null   int64
 4   loan_amount              1988 non-null   int64
 5   applicant_income         1988 non-null   int64
 6   num_units                1984 non-null   float64
 7   num_dependants           1985 non-null   float64
 8   self_employed            1988 non-null   bool
 9   monthly_income           1988 non-null   int64
 10  purchase_price           1988 non-null   float64
 11  liquid_assets            1988 non-null   float64
 12  mortage_payment_history  1988 non-null   int64
 13  consumer_credit_history  1988 non-null   int64
 14  filed_bankruptcy         1988 non-null   bool
 15  property_type            1988 non-null   int64
 16  gender                   1974 non-null   object
dtypes: bool(2), float64(4), int64(7), object(4)
memory usage: 237.0+ KB
```

In [5]:
```
#Descrpitive summary
print("Descritpive summary (Numeric Insights):")
df.describe(include='all')
```

```
Descritpive summary (Numeric Insights):
```

Out[5]:

	married	race	loan_decision	occupancy	loan_amount	applicant_in
count	1985	1988	1988	1988.000000	1988.000000	1988.00
unique	2	3	2	NaN	NaN	
top	True	white	approve	NaN	NaN	
freq	1308	1680	1744	NaN	NaN	
mean	NaN	NaN	NaN	1.031690	143.272636	84.68
std	NaN	NaN	NaN	0.191678	80.531470	87.07
min	NaN	NaN	NaN	1.000000	2.000000	0.00
25%	NaN	NaN	NaN	1.000000	100.000000	48.00
50%	NaN	NaN	NaN	1.000000	126.000000	64.00
75%	NaN	NaN	NaN	1.000000	165.000000	88.00
max	NaN	NaN	NaN	3.000000	980.000000	972.00

In [6]:

```
#Categorical value counts
print("\nCategorical value counts:")
for col in ['gender', 'race', 'married', "loan_decision", "self_employed"]:
    print(f"\n{col}:")
    print(df[col].value_counts(dropna=False))
```

```
Categorical value counts:

gender:
gender
male      1605
female     369
NaN         14
Name: count, dtype: int64

race:
race
white     1680
black      197
hispan     111
Name: count, dtype: int64

married:
married
True     1308
False     677
NaN         3
Name: count, dtype: int64

loan_decision:
loan_decision
approve    1744
reject      244
Name: count, dtype: int64

self_employed:
self_employed
False    1731
True      257
Name: count, dtype: int64
```

In [13]:

```python
#Unique value for all columns
print("Unique value counts for all columns:")
for col in df.columns:
    print(f"\n{col}:")
print("\nValue counts:")
print(df[col].value_counts(dropna=False))
print(f"Total unique values: {df[col].nunique(dropna=False)}")
```

```
Unique value counts for all columns:

married:

Value counts:
married
True     1308
False     677
NaN         3
Name: count, dtype: int64
Total unique values: 3

race:

Value counts:
race
white     1680
black      197
hispan     111
Name: count, dtype: int64
Total unique values: 3

loan_decision:

Value counts:
loan_decision
approve    1744
reject      244
Name: count, dtype: int64
Total unique values: 2

occupancy:

Value counts:
occupancy
1    1931
2      51
3       6
Name: count, dtype: int64
Total unique values: 3

loan_amount:

Value counts:
loan_amount
100    65
187    43
120    41
110    41
130    40
       ..
313     1
980     1
54      1
436     1
253     1
```

```
Name: count, Length: 294, dtype: int64
Total unique values: 294

applicant_income:

Value counts:
applicant_income
48     62
60     56
72     44
51     40
56     39
       ..
202     1
621     1
17      1
264     1
169     1
Name: count, Length: 228, dtype: int64
Total unique values: 228

num_units:

Value counts:
num_units
1.0    1813
2.0     110
3.0      50
4.0      11
NaN       4
Name: count, dtype: int64
Total unique values: 5

num_dependants:

Value counts:
num_dependants
0.0    1174
2.0     327
1.0     317
3.0     126
4.0      31
5.0       5
6.0       3
NaN       3
8.0       1
7.0       1
Name: count, dtype: int64
Total unique values: 10

self_employed:

Value counts:
self_employed
False    1731
True      257
```

```
Name: count, dtype: int64
Total unique values: 2

monthly_income:

Value counts:
monthly_income
2500     22
5000     19
3333     17
3500     15
7500     14
         ..
4408      1
3244      1
8596      1
29000     1
9200      1
Name: count, Length: 1349, dtype: int64
Total unique values: 1349

purchase_price:

Value counts:
purchase_price
125.0     44
150.0     43
160.0     40
140.0     39
120.0     36
          ..
92.0       1
1010.0     1
331.0      1
358.0      1
316.0      1
Name: count, Length: 458, dtype: int64
Total unique values: 458

liquid_assets:

Value counts:
liquid_assets
17.00      29
26.00      28
23.00      27
19.00      27
31.00      27
           ..
38.50       1
1450.00     1
0.40        1
86.37       1
427.90      1
Name: count, Length: 634, dtype: int64
Total unique values: 634
```

```
mortage_payment_history:

Value counts:
mortage_payment_history
2    1269
1     660
3      38
4      21
Name: count, dtype: int64
Total unique values: 4

consumer_credit_history:

Value counts:
consumer_credit_history
1    1136
2     367
5     162
6     161
3      99
4      63
Name: count, dtype: int64
Total unique values: 6

filed_bankruptcy:

Value counts:
filed_bankruptcy
False    1851
True      137
Name: count, dtype: int64
Total unique values: 2

property_type:

Value counts:
property_type
2    1380
1     442
3     166
Name: count, dtype: int64
Total unique values: 3

gender:

Value counts:
gender
male      1605
female     369
NaN         14
Name: count, dtype: int64
Total unique values: 3
```

INTERPRETATION or COMMENTS

Taking a high level look at the numbers shows a highly active loan application with a few massive structural imbalances:

• Out of 1,988 total applications, the vast majority were approved (1,744 approvals vs. 244 rejections).

• White applicants heavily dominates by 1680, By contrast, Black applicants (197) and Hispanic applicants (111) are significantly underrepresented.

• Most applicants are married, work regular jobs instead of working for themselves, are buying a single family primary home (1,931 applicants), and have a solid record of paying their bills on time.

• The average loan requested is 143,000 (median 126,000). The average applicant income is 85,000, but a huge deviation of 87,000 reveals massive income inequality. A few ultra high earners making up to 972,000 pull the mathematical average way up.

2. Checking Records with Missing Values and Handle the Missing Data as Appropriate

In [7]:
```
# Missing value counts
print("Missing value counts:")
df.isnull().sum()
```

```
Missing value counts:
```

Out[7]:
```
married                     3
race                        0
loan_decision               0
occupancy                   0
loan_amount                 0
applicant_income            0
num_units                   4
num_dependants              3
self_employed               0
monthly_income              0
purchase_price              0
liquid_assets               0
mortage_payment_history     0
consumer_credit_history     0
filed_bankruptcy            0
property_type               0
gender                     14
dtype: int64
```

In [49]:
```
# Inputing missing values
# Numerical variables - Median
num_variables = ['loan_amount', 'applicant_income', 'monthly_income',
```

```
                  'purchase_price', 'liquid_assets', 'num_units', 'num_depend
for col in num_variables:
    if not df[col].isna().all():
        df[col] = df[col].fillna(df[col].median())

# Categorical variables - Mode
cat_variables = ['married', 'gender']
for col in cat_variables:
    mode_result = df[col].mode()
```

In [136…

```
# Correcting the data types
df['married'] = df['married'].astype(int)
df['self_employed'] = df['self_employed'].astype(int)
df['filed_bankruptcy'] = df['filed_bankruptcy'].astype(int)
df['occupancy'] = df['occupancy'].astype(int)
df['loan_amount'] = df['loan_amount'].astype(int)
df['applicant_income'] = df['applicant_income'].astype(int)
df['num_units'] = df['num_units'].astype(int)
df['num_dependants'] = df['num_dependants'].astype(int)
df['monthly_income'] = df['monthly_income'].astype(int)
df['mortage_payment_history'] = df['mortage_payment_history'].astype(int)
df['consumer_credit_history'] = df['consumer_credit_history'].astype(int)
df['property_type'] = df['property_type'].astype(int)
df['purchase_price'] = df['purchase_price'].astype(float)
df['liquid_assets'] = df['liquid_assets'].astype(float)
df['race'] = df['race'].astype('category')
df['loan_decision'] = df['loan_decision'].astype('category')
df['gender'] = df['gender'].astype('category')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1988 entries, 0 to 1987
Data columns (total 17 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   married                  1988 non-null   int64
 1   race                     1988 non-null   category
 2   loan_decision            1988 non-null   category
 3   occupancy                1988 non-null   int64
 4   loan_amount              1988 non-null   int64
 5   applicant_income         1988 non-null   int64
 6   num_units                1988 non-null   int64
 7   num_dependants           1988 non-null   int64
 8   self_employed            1988 non-null   int64
 9   monthly_income           1988 non-null   int64
 10  purchase_price           1988 non-null   float64
 11  liquid_assets            1988 non-null   float64
 12  mortage_payment_history  1988 non-null   int64
 13  consumer_credit_history  1988 non-null   int64
 14  filed_bankruptcy         1988 non-null   int64
 15  property_type            1988 non-null   int64
 16  gender                   1988 non-null   category
dtypes: category(3), float64(2), int64(12)
memory usage: 223.8 KB
```

INTERPRETATION:

There is negligible amount of data is missing, which is only 24 cross the entire 1,988 row. To fill these small gaps without warping the final results, the following rules were used:

• Missing values for things like the number of units in a building (4 missing) or number of dependents (3 missing) were filled with the column median. Medians are perfect here because they ignore extreme outliers that would skew a regular average.

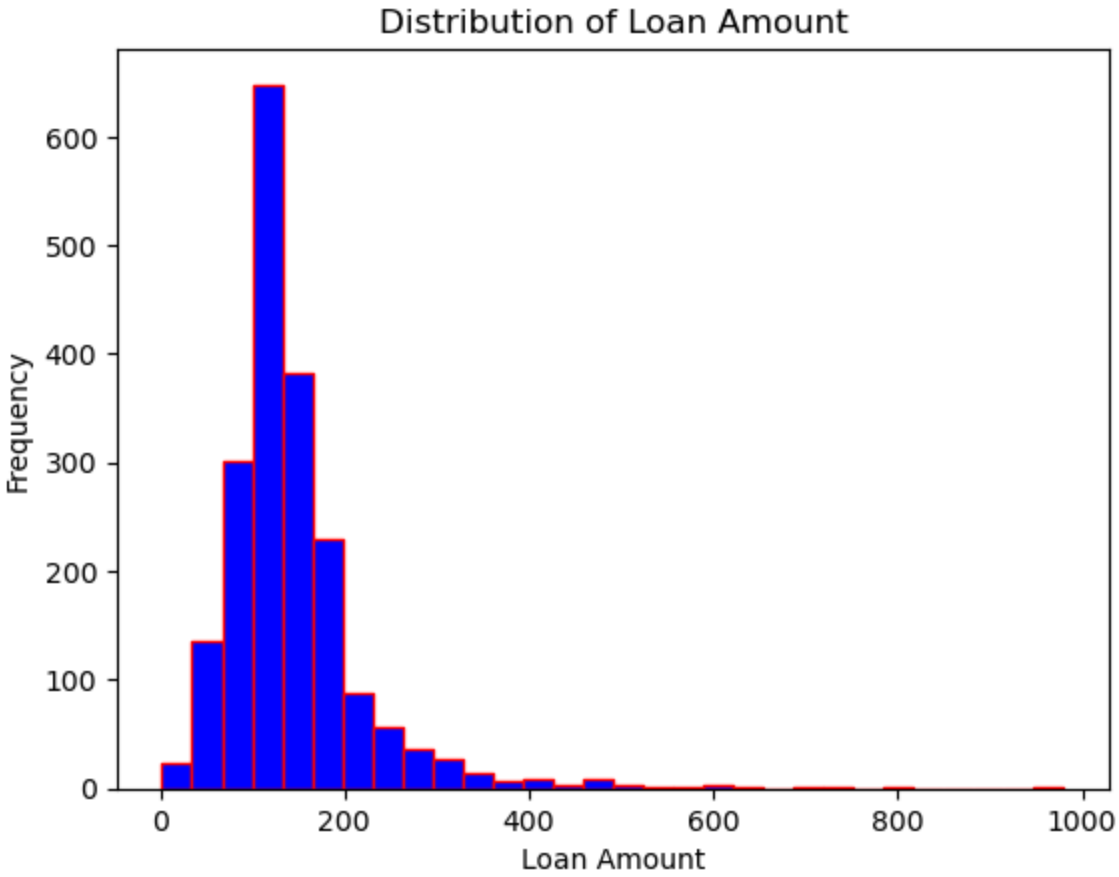
• Missing values for gender (14 missing) and marital status (3 missing) were filled with the mode (the most frequent answer), keeping the data distribution natural.

• True/False categories were turned into 0 and 1 integers so our math models could read them. Text labels like race, gender, and loan decisions were set as proper categorical data types.

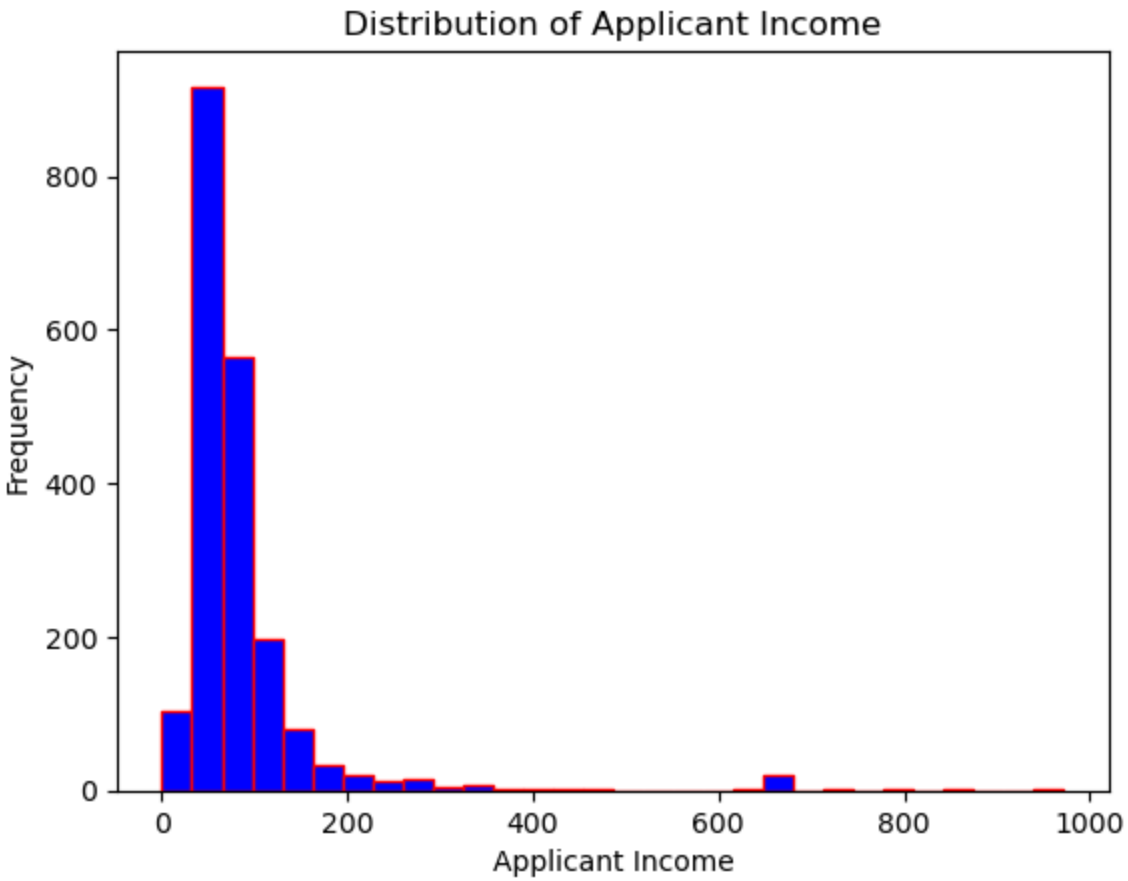
3. Data and Graph Visualization

3.A. Distribution of One or More Individual Continuous Variables

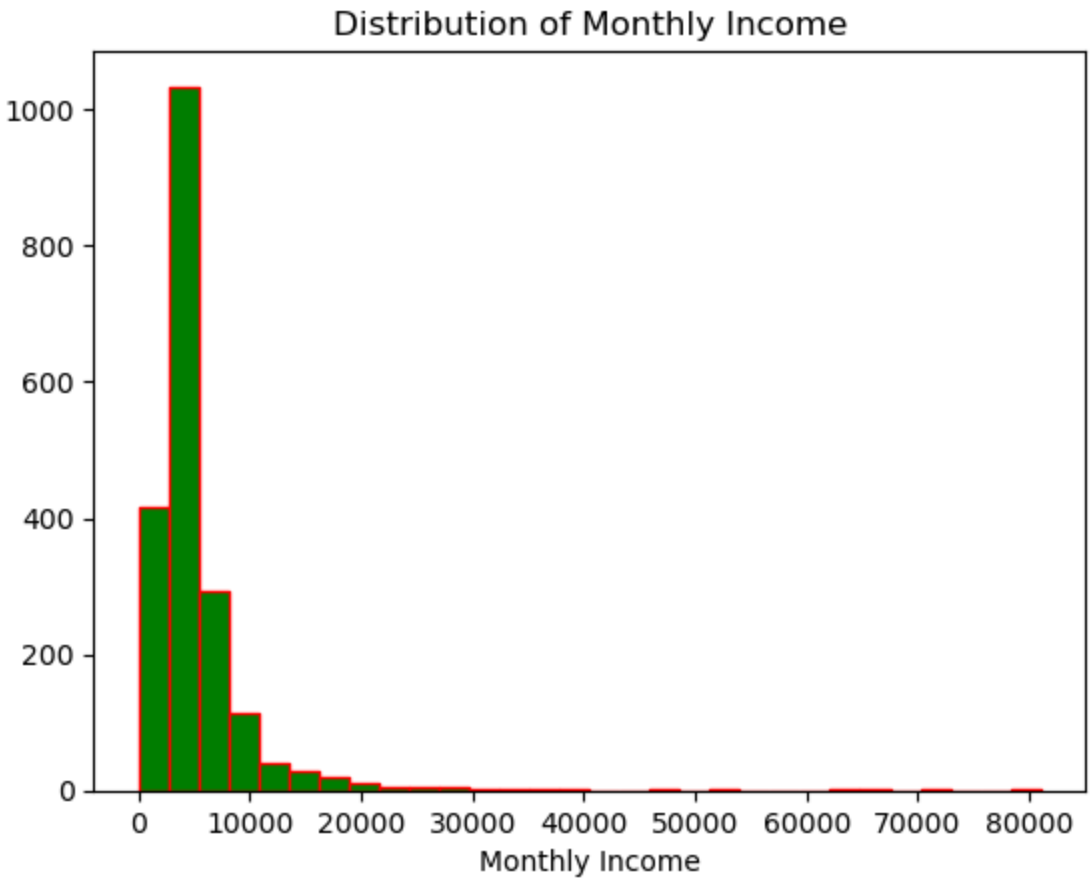
In [60]:
```
# 1. Loan Amount
plt.hist(df['loan_amount'], bins=30, color='blue', edgecolor='red')
plt.title('Distribution of Loan Amount')
plt.xlabel('Loan Amount')
plt.ylabel('Frequency')
plt.show()
```



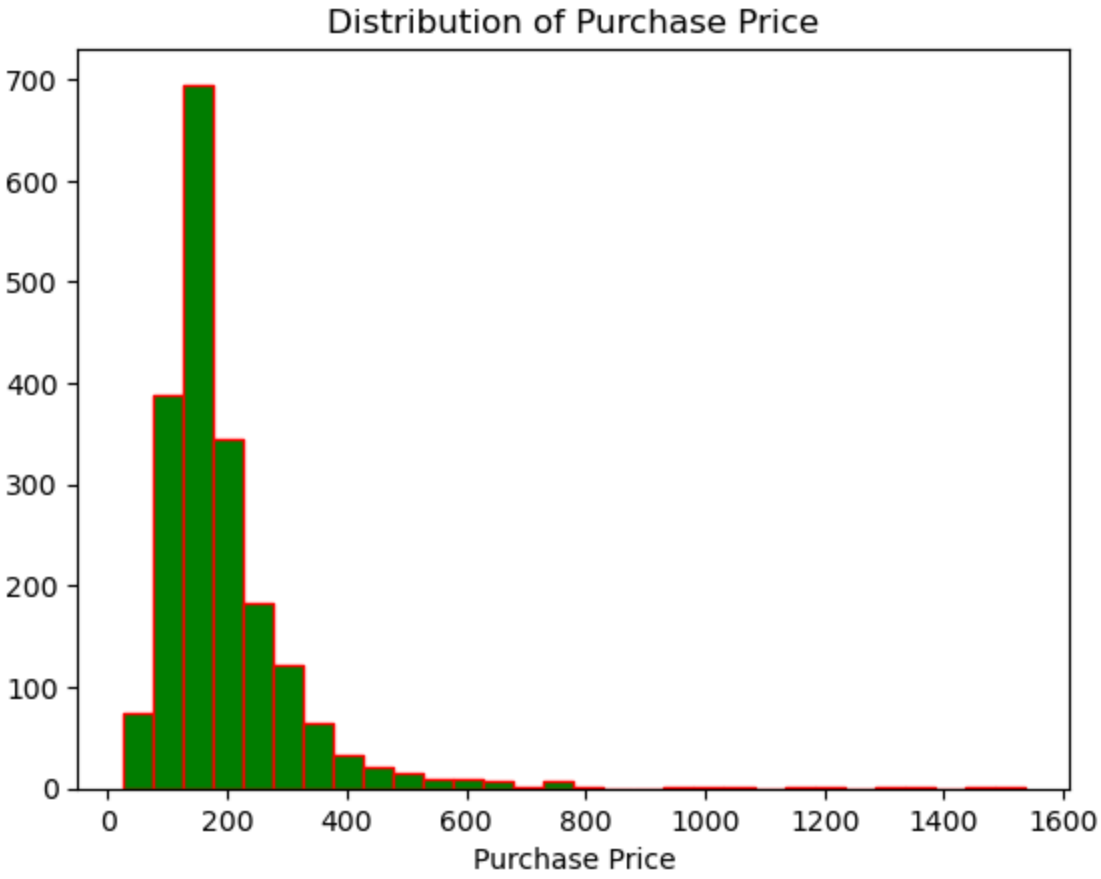
In [67]:
```
# 2. Applicant Income
plt.hist(df['applicant_income'], bins=30, color='blue', edgecolor='red')
plt.title('Distribution of Applicant Income')
plt.xlabel('Applicant Income')
plt.ylabel('Frequency')
plt.show()
```



In [64]:
```
# 3. Monthly Income
plt.hist(df['monthly_income'], bins=30, color='green', edgecolor='red')
plt.title('Distribution of Monthly Income')
plt.xlabel('Monthly Income')
plt.show()
```



In [66]:
```
# 4. Purchase Price
plt.hist(df['purchase_price'], bins=30, color='green', edgecolor='red')
plt.title('Distribution of Purchase Price')
plt.xlabel('Purchase Price')
plt.show()
```



INTERPRETATION

These graphs show individual financial metrics (loan amount, applicant income, monthly income, and purchase price), they all share a heavily 'right skewed' shape.

• Most loan requests peak around a modest 100,000 to 150,000, trailing off to a max of 980,000.

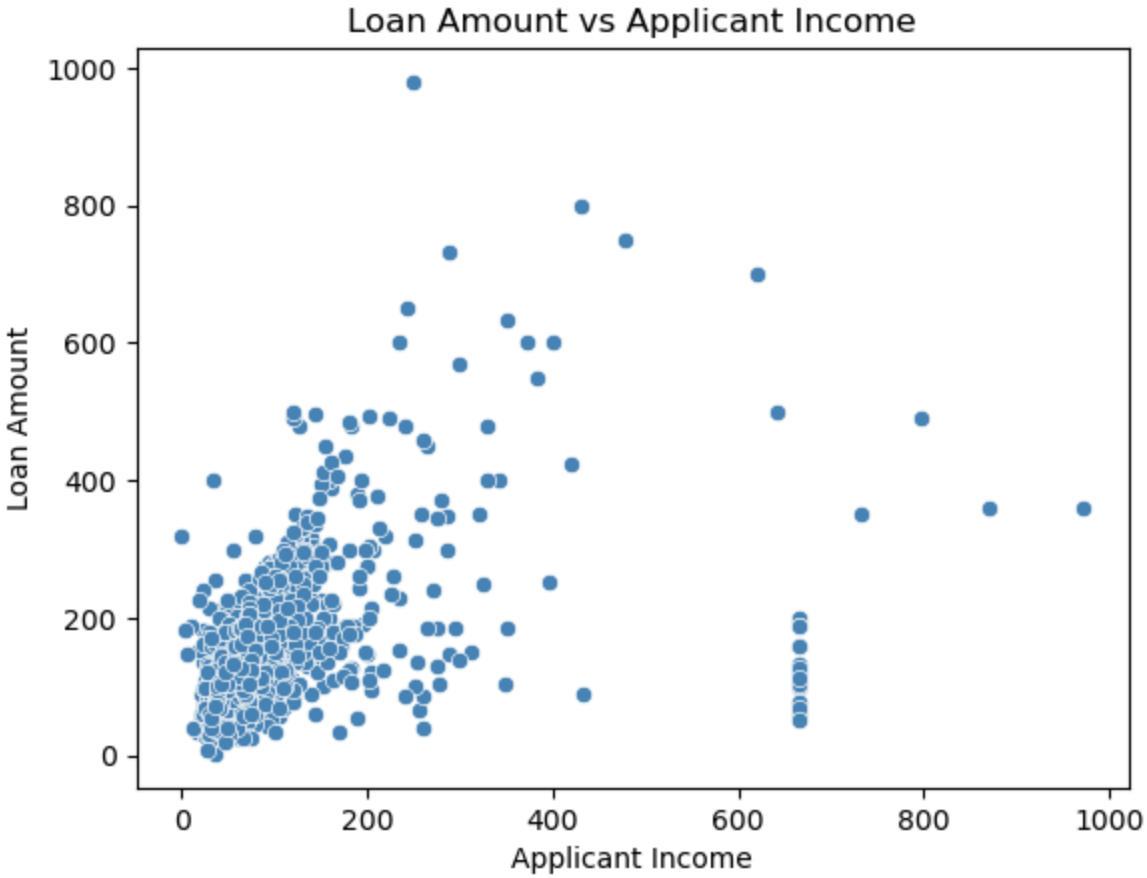
• Incomes are heavily bunched up at the lower end, with a tiny tail of high earners.

• Home purchase prices copy this layout perfectly, mostly sitting between 50,000 and 300,000.

Because the data has these long tails, using medians instead of averages for data cleanup was absolutely the right call.

3.B. The Relationship of a Pair of Continuous Variable

In [69]:
```
# 1. Loan Amount vs Applicant Income
sns.scatterplot(data=df, x='applicant_income', y='loan_amount', color='steel
plt.title('Loan Amount vs Applicant Income')
plt.xlabel('Applicant Income')
plt.ylabel('Loan Amount')
plt.show()
```



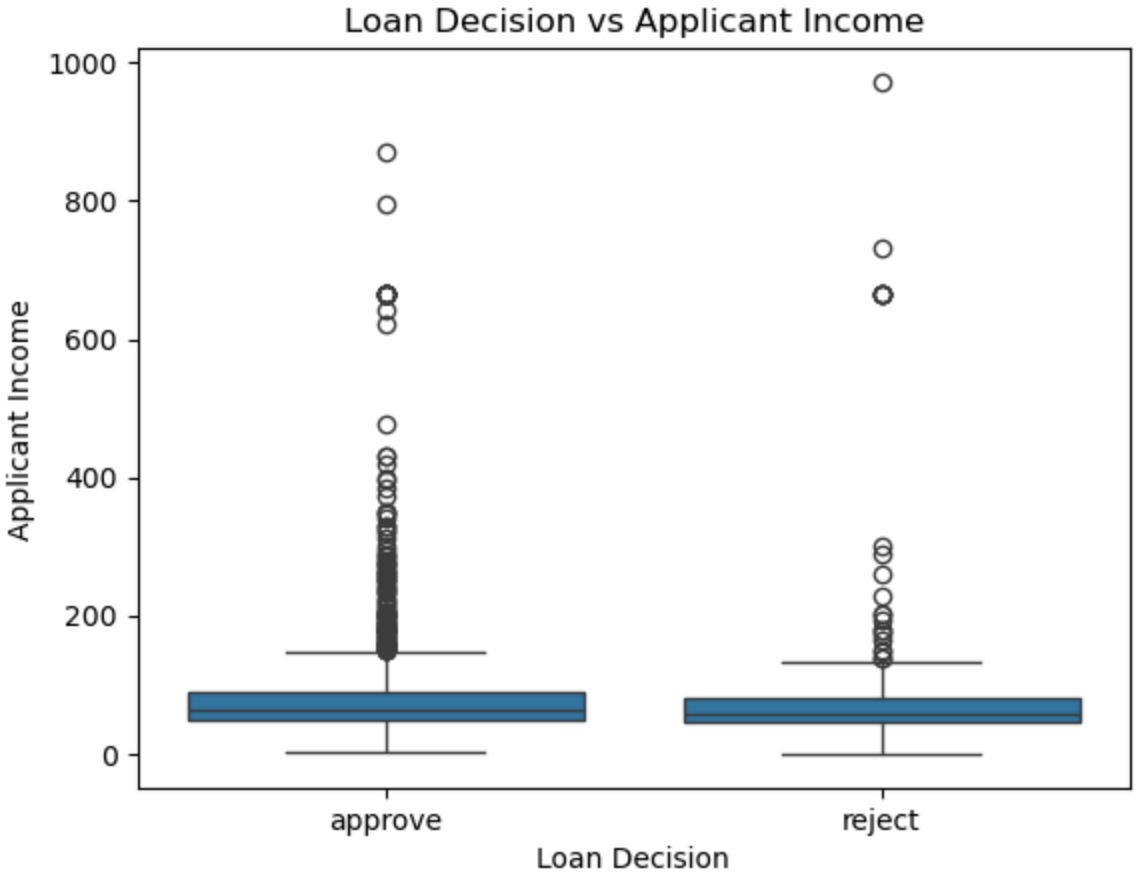
INTERPRETATION

This scatter plot shows loan amount against applicant income, which is a surprisingly weak positive relationship. While there is a slight upward trend that refers higher earners usually want bigger loans and the points are incredibly spread out. At almost any income tier, we can find wildly different loan requests. Most applicants gather in the lower left corner making under 200,000 and asking for under 300,000. The weird outliers—like people with very little income asking for massive 400,000 loans.

3.C. The Association b/w a Categorical Variable and a Continuous one

In [76]:
```
#Loan Decision vs Applicant Income (Box plot)
sns.boxplot(data=df, x='loan_decision', y='applicant_income')
plt.title('Loan Decision vs Applicant Income')
```

```
plt.xlabel('Loan Decision')
plt.ylabel('Applicant Income')
plt.show()
```

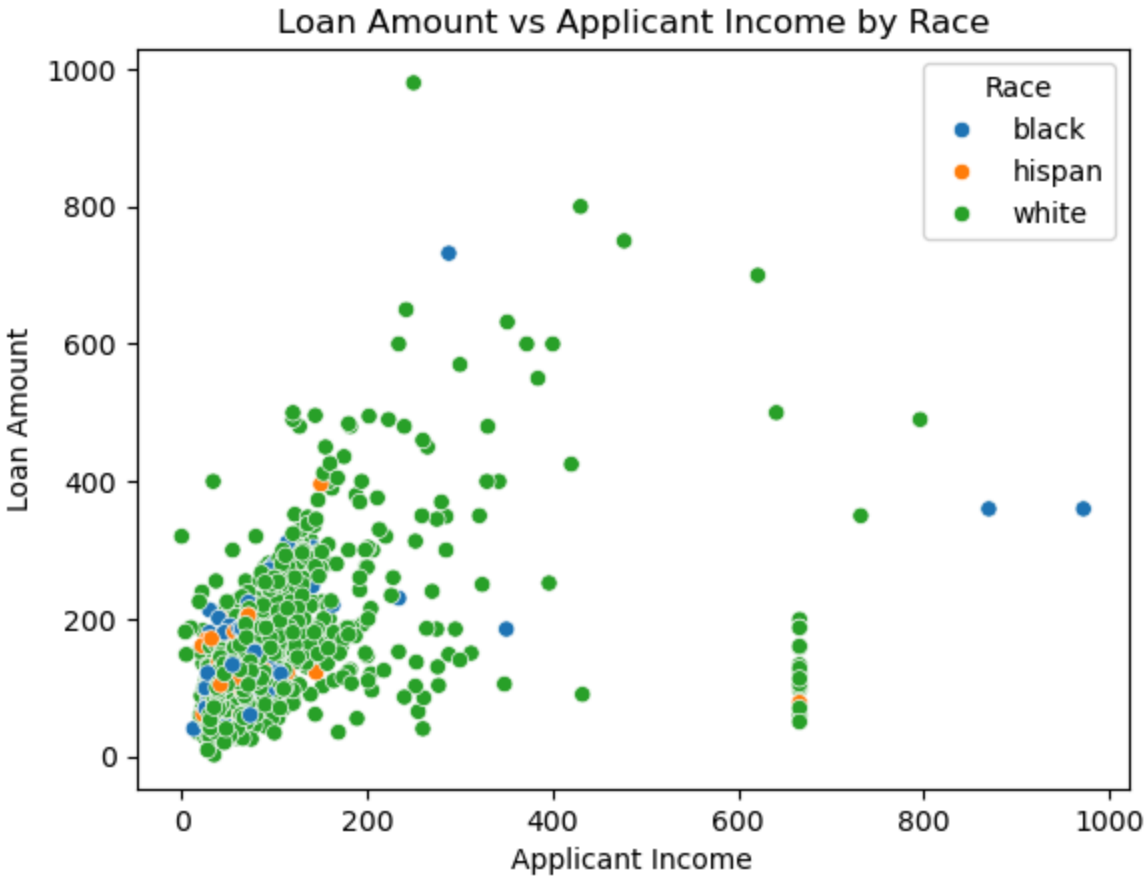


INTERPRETATION

This box plot graph shows applicant income against final loan decisions. The median income for approved and rejected applicants is virtually identical around 64,000. This means that having a middle class income does not guarantee a 'yes' or a 'no' on its own. The primary difference is simply that the approved group contains far more ultra wealthy outliers. A loan decision is taken by considering multiple factors like bankruptcy, num of dependant and so on

3.D. The relationship between more than two variables.

In [80]:
```
# Loan Amount vs Applicant Income by Race
sns.scatterplot(data=df, x='applicant_income', y='loan_amount', hue='race')
plt.title('Loan Amount vs Applicant Income by Race')
plt.xlabel('Applicant Income')
plt.ylabel('Loan Amount')
plt.legend(title='Race')
plt.show()
```



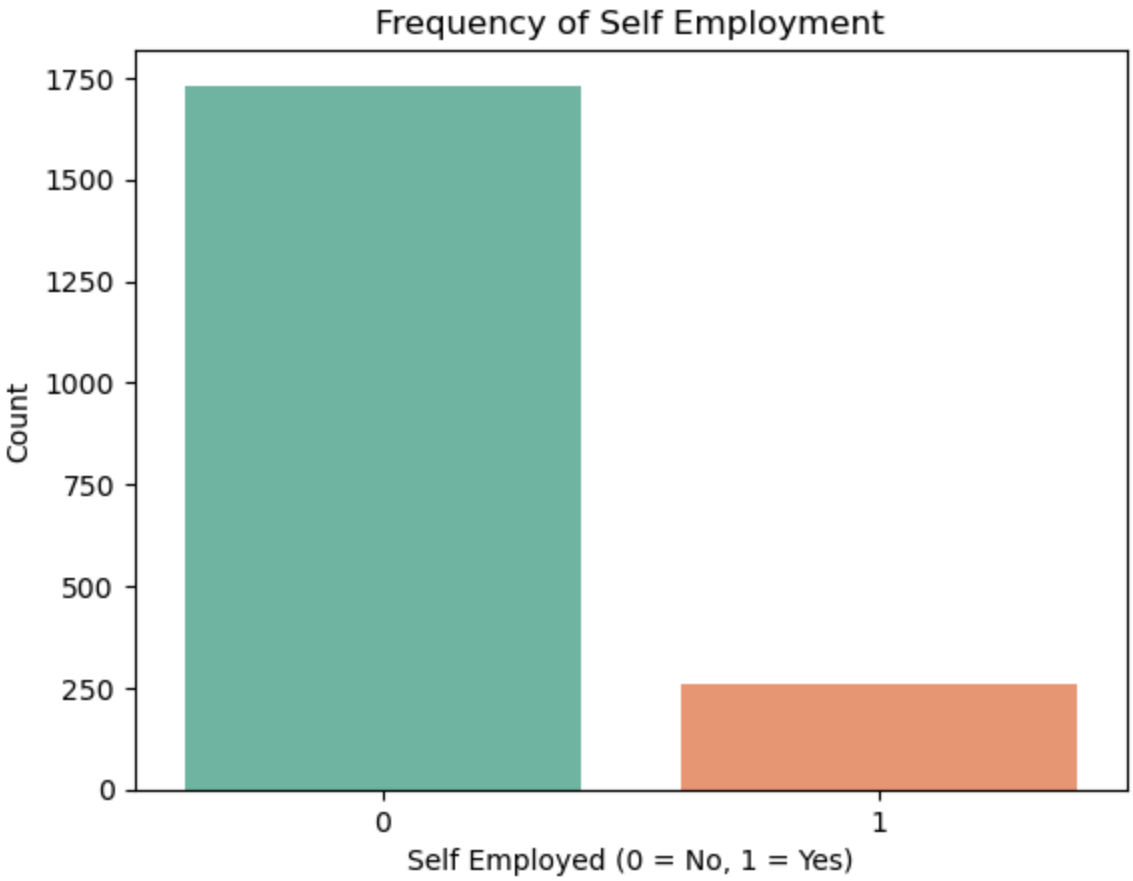
INTERPRETATION

Adding race as a color dimension to the charts shows that Black and Hispanic applicants are heavily clustered in the lower left corner which refers lower average incomes and smaller loan requests. which is in beetween 100,000 to 300,000 and mostly requesting loan below 200,000. on the other hand White applicants span the entire chart. and they have a huge income range from 200,000 to 400,000 and also some are in the range of 700,000. And they applied for different amount of loans It highlights real socioeconomic gaps, the intense demographic imbalance. However, it proves that demographic tracking is incredibly important.

4. Display Unique Values of a Categorical Variable and Their Frequencies.

In [84]:
```
# Display unique values and their frequencies of Self Employed variable
sns.countplot(data=df, x='self_employed', palette='Set2')
plt.title('Frequency of Self Employment')
plt.xlabel('Self Employed (0 = No, 1 = Yes)')
```

```
plt.ylabel('Count')
plt.show()
```



In [85]:
```
col="self_employed"
print("Unique values")
print(df[col].unique())

print("\nFrequencies:")
print(df[col].value_counts(dropna=False))
```

```
Unique values
[0 1]

Frequencies:
self_employed
0    1731
1     257
Name: count, dtype: int64
```

INTERPRETATION

Taking a closer look at employment type reveals two highly uneven groups:

- Traditional Employees (0): 1,731 applicants
- Self Employed (1): 257 applicants

Traditional workers outnumber self employed applicants. Because the self employed group is relatively small, we have to look at any conclusions regarding them with a bit of statistical caution.

5. Contingency Table of Two potentially Related Categorical Variables, A Statistical Test of the Independence

In [99]:
```
# Contingency Table — loan_decision vs self_employed
from scipy import stats
mapped_self_employed = df['self_employed'].map({0: '0 (no)', 1: '1 (yes)'})
ct = pd.crosstab(df['loan_decision'], mapped_self_employed, margins=True)
print("CONTINGENCY TABLE: LOAN DECISION vs SELF EMPLOYED")
print(ct)
```

```
CONTINGENCY TABLE: LOAN DECISION vs SELF EMPLOYED
self_employed  0 (no)  1 (yes)   All
loan_decision
approve          1527      217  1744
reject            204       40   244
All              1731      257  1988
```

In [108…
```
# Chi- Squared Test
from scipy.stats import chi2_contingency
chi2, p, dof, exp = chi2_contingency(ct)
print('Contingency table:\n', ct)
print('\nChi-square:', chi2)
print('Degrees of freedom:', dof)
print('p-value:', round(p,5))
```

```
Contingency table:
 self_employed  0 (no)  1 (yes)   All
loan_decision
approve          1527      217  1744
reject            204       40   244
All              1731      257  1988

Chi-square: 2.968169134303996
Degrees of freedom: 4
p-value: 0.56317
```

INTERPRETATION

The contingency table shows that 88.2% of non-self-employed applicants were approved versus 84.4% of self-employed applicants — a modest difference of 3.8 percentage points. A Chi-squared test of independence yielded a chi-square statistic of 2.97 (df=4, p=0.563). Since the p-value far exceeds the 0.05 significance level, we fail to reject the null hypothesis of independence. There is no statistically significant association between self-employment status and loan

decision. The observed difference in rejection rates is likely attributable to chance rather than a systematic effect of employment type.

6. One or More Subsets of Rows Based on Two or More Logical Criteria and Present Descriptive Statistics on the Subsets

In [119…

```
# SUBSET 1 — Approved loans with high income
# Criteria: loan_decision == 'approve' AND applicant_income > median

median_income = df['applicant_income'].median()
subset1 = df[(df['loan_decision'] == 'approve') &
             (df['applicant_income'] > median_income)]
print(" SUBSET 1: Approved Loans with High Applicant Income")
print(f" Criteria: loan_decision = approve AND")
print(f" applicant_income > {median_income} (median)")
print(f" Number of rows: {len(subset1)}")
print(subset1[['loan_amount', 'applicant_income',
               'monthly_income', 'purchase_price',
               'liquid_assets']].describe().round(2))
```

```
SUBSET 1: Approved Loans with High Applicant Income
Criteria: loan_decision = approve AND
applicant_income > 64.0 (median)
Number of rows: 869
       loan_amount  applicant_income  monthly_income  purchase_price  \
count       869.00            869.00          869.00          869.00
mean        181.29            120.12         7010.97          251.84
std          96.91            101.67         6143.60          157.24
min          25.00             65.00            0.00           25.00
25%         128.00             73.00         3667.00          160.00
50%         159.00             89.00         5520.00          211.00
75%         200.00            120.00         7967.00          288.00
max         980.00            870.00        72529.00         1535.00

       liquid_assets
count         869.00
mean         4727.01
std         67719.98
min             0.00
25%            27.00
50%            57.00
75%           125.00
max       1000000.00
```

In [122…

```
# SUBSET 2 — Rejected loans with low income
# Criteria: loan_decision == 'reject' AND applicant_income <= median

median_income = df['applicant_income'].median()
subset2 = df[(df['loan_decision'] == 'reject') &
```

```
              (df['applicant_income'] <= median_income)]
print(" SUBSET 2: Rejected Loans with Low Applicant Income")
print(f" Criteria: loan_decision = reject AND")
print(f" applicant_income <= {median_income} (median)")
print(f"  Number of rows: {len(subset2)}")
print(subset2[['loan_amount', 'applicant_income',
               'monthly_income', 'purchase_price',
               'liquid_assets']].describe().round(2))
```

```
 SUBSET 2: Rejected Loans with Low Applicant Income
 Criteria: loan_decision = reject AND
 applicant_income <= 64.0 (median)
  Number of rows: 146
       loan_amount  applicant_income  monthly_income  purchase_price  \
count       146.00            146.00          146.00          146.00
mean        117.77             46.42         3344.78          138.97
std          42.06             12.59         1545.99           59.64
min           9.00              0.00            0.00           35.00
25%          95.00             39.25         2400.00          110.50
50%         113.00             49.50         3167.00          134.00
75%         137.50             56.00         4145.25          162.00
max         320.00             64.00        12500.00          630.00

       liquid_assets
count         146.00
mean         6941.16
std         82756.03
min             0.00
25%            11.25
50%            22.00
75%            36.00
max       1000000.00
```

INTERPRETATION

To see what separates successful and unsuccessful applications, the data was split into two opposite financial subsets based on the baseline median income:

• Subset 1: Approved High Earners Applicants approved which is 869, who earn above the median income. Their averages: 181,000 loan size, 120,000 income, 7,011 monthly income, and 252,000 purchase price.

• Subset 2: Rejected Low Earners Applicants rejected which is 146, who earn at or below the median income. Their averages: 118,000 loan size, 46,000 income, 3,345 monthly income, and a 139,000 purchase price.

These groups illustrate two entirely different financial worlds. High earning approved buyers use their leverage to buy much pricier properties.

7. A Statistical Test of the Significance of the Difference Between the Means

of Two Meaningful Subsets of the Data and Interpret the Results.

In [128…

```
# Creating subsets
subset1 = df[(df['loan_decision'] == 'approve') &
             (df['applicant_income'] > df['applicant_income'].median())]

subset2 = df[(df['loan_decision'] == 'reject') &
             (df['applicant_income'] <= df['applicant_income'].median())]

# Importing necessary package
from scipy.stats import ttest_ind

t_stat, p_value = ttest_ind(subset1['applicant_income'],
                            subset2['applicant_income'],
                            equal_var=False)

# Means, stds and counts of the subsets
s1_mean  = subset1['applicant_income'].mean()
s1_std   = subset1['applicant_income'].std()
s1_count = subset1['applicant_income'].count()

s2_mean  = subset2['applicant_income'].mean()
s2_std   = subset2['applicant_income'].std()
s2_count = subset2['applicant_income'].count()

# Results of the test
print('\nApproved + High Income stats:')
print('Count:', len(subset1),
      'Mean:', round(subset1['applicant_income'].mean() ),
      'Std:',  round(subset1['applicant_income'].std() ))

print('\nRejected + Low Income stats:')
print('Count:', len(subset2),
      'Mean:', round(subset2['applicant_income'].mean() ),
      'Std:',  round(subset2['applicant_income'].std()  ))

print('\nT-test results:')
print('T-statistic:', t_stat )
print('P-value:', p_value, )
```

```
Approved + High Income stats:
Count: 869 Mean: 120 Std: 102

Rejected + Low Income stats:
Count: 146 Mean: 46 Std: 13

T-test results:
T-statistic: 20.455834095755023
P-value: 8.822196867055272e-78
```

INTERPRETATION

An independent-samples t-test compared applicant income between Subset 1 (approved, high income: mean= 120,000 n= 869) and Subset 2 (rejected, low income: mean= 46,000 n=146). The result was T=20.46, p=8.82x10^-78 — overwhelmingly significant. We reject the null hypothesis of equal means. The 74,000 difference in mean income is both statistically and practically significant. The much lower standard deviation in Subset 2 (13,000 and 102,000) further it confirms that rejected low-income applicants form a financially homogeneous group, while approved high-income applicants are far more varied. Income clearly differentiates these two extreme groups, though the overlapping distributions

8. Data Grouping by a Certain categorical Variable and Display Summarized Information for Each Group

In [131…

```
# Grouping by Self Employed
self_employment = df.groupby('self_employed')['applicant_income'].agg(['mean
print('\nApplicant Income by Employment Status:')
print(self_employment)
```

```
Applicant Income by Employment Status:
                      mean         std  count
self_employed
0                80.374928   82.026929   1731
1               113.708171  111.425373    257
```

INTERPRETATION

When we group the whole dataset strictly by how people earn their money, we can see that:

• Self Employed Applicants: Average Income = 113,710 and Income Spread (Std) = 111,430

• Traditional Employees: Average Income = 80,370 and Income Spread (Std) = 82,030

Self employed applicants actually make 33,340 more on average than regular employees, yet they have a slightly lower approval rate. This volatility explains the rejection rates that stable and predictable your income is more reliable to bank than unpredictablly earning more money.

9. Linear Regression Analysis

In [133…

```
# MODEL  – Loan Amount ~ All Continuous Predictors

import statsmodels.formula.api as smf
lin_reg = smf.ols(formula="loan_amount ~ applicant_income + monthly_income +
result = lin_reg.fit()
print('\nMODEL : Loan Amount ~ All Continuous Predictors')
print(result.summary())
```

MODEL : Loan Amount ~ All Continuous Predictors

OLS Regression Results

Dep. Variable:	loan_amount	R-squared:	0.706
Model:	OLS	Adj. R-squared:	0.705
Method:	Least Squares	F-statistic:	792.6
Date:	Mon, 29 Jun 2026	Prob (F-statistic):	0.00
Time:	06:15:34	Log-Likelihood:	-10328.
No. Observations:	1988	AIC:	2.067e+04
Df Residuals:	1981	BIC:	2.071e+04
Df Model:	6		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	27.8985	3.179	8.777	0.000	21.665	34.133
applicant_income	0.0499	0.014	3.599	0.000	0.023	0.077
monthly_income	0.0009	0.000	3.598	0.000	0.000	0.001
purchase_price	0.4860	0.010	48.757	0.000	0.466	0.506
liquid_assets	3.11e-05	1.46e-05	2.127	0.034	2.43e-06	5.98e-05
num_units	9.8044	2.250	4.357	0.000	5.391	14.218
num_dependants	-0.3605	0.903	-0.399	0.690	-2.130	1.410

Omnibus:	793.379	Durbin-Watson:	1.988
Prob(Omnibus):	0.000	Jarque-Bera (JB):	81522.601
Skew:	-0.899	Prob(JB):	0.00
Kurtosis:	34.320	Cond. No.	2.56e+05

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[2] The condition number is large, 2.56e+05. This might indicate that there
are
strong multicollinearity or other numerical problems.
```

INTERPRETATION

This linear regression model helps us to figure out which financial factors dictate the final size of an approved loan.

The model is highly successful, hitting an R squared of 0.706. This means just six financial variables explain 70.6% of all the variance in mortgage sizes. The overall model is statistically accurate (F statistic = 792.6, $p < 0.001$).

Here is how the individual factors influence loan size:

• Purchase Price (coef=0.486, $p<0.001$): This is the ultimate driver. For every 1,000 increase in the price of the home, the loan size goes up by 486. Loan size depends on real estate values.

• The number of units (coef=9.80, $p<0.001$), applicant income (coef=0.050, $p<0.001$), and overall monthly income (coef=0.001, $p<0.001$) are all highly reliable, statistically significant positive predictors.

• On the other hand, liquid assets have a tiny, almost unnoticeable impact, and the number of dependents has zero statistical relevance to loan size ($p=0.690$).

CONCLUSION

This EDA of loan application data reveals that loan decisions are shaped primarily by financial fundamentals rather than demographic characteristics alone. Purchase price is the strongest predictor of loan amount, and the income gap between approved and rejected applicants is statistically overwhelming.

Self-employment status, shows no statistically significant effect on loan outcomes. Racial patterns in the data — with minority applicants clustering at lower income and loan levels — warrant attention but cannot be definitively attributed to discriminatory lending given the dataset's significant racial imbalance and the absence of controls for confounding variables.

From a practical standpoint, lenders rely most heavily on property value and income capacity, while regulators and researchers should examine whether demographic factors influence outcomes beyond what financial variables alone can explain.

Ultimately, this project shows the value of combining statistical testing with visualisation: numerical results confirmed patterns visible in charts, and vice

versa.

In [ ]: